Toronto Metropolitan University

**Department of Electrical, Computer & Biomedical Engineering**
Faculty of Engineering & Architectural Science

| | |
|---|---|
| **Course Title:** | **Fundamentals of Data Engineering** |
| **Course Number:** | **COE848** |
| **Semester/Year (e.g. F2016)** | **W2026** |

| | |
|---|---|
| **Instructor:** | **Dr. Yasaman Ahmadiadli** |
| **Lab Report Number** | **Lab 4** |
| **Lab Title** | **Lab 4: Manipulating Data** |

| | |
|---|---|
| **Due Date** | **March 12, 2026** |
| **Submission Date** | **March 12, 2026** |


| Last Name | First Name | Student Number | Section | Signature* |
|---|---|---|---|---|
| **Bhuiyan** | **Hasib** | *******9402** | **03** |  |


*By signing above you attest that you have contributed to this written lab report and confirm that all work you have contributed to this lab report is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: http://www.ryerson.ca/senate/current/pol60.pdf

```
PRAGMA foreign_keys=OFF;
BEGIN TRANSACTION;
CREATE TABLE Restaurant (
    restaurant_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    address TEXT NOT NULL,
    phone_number TEXT NOT NULL,
    cuisine_type TEXT,
    opening_hours TEXT,
    Closing_hours TEXT,
    total_capacity INTEGER
);
INSERT INTO Restaurant VALUES(1,'Villa Grill','123 King St
Toronto','4165551000','Mediterranean','10:00','22:00',80);
INSERT INTO Restaurant VALUES(2,'Sunset Sushi','22 Queen St
Toronto','4165551001','Japanese','11:00','23:00',60);
INSERT INTO Restaurant VALUES(3,'Pasta Palace','77 Dundas St
Toronto','4165551002','Italian','10:00','22:00',70);
INSERT INTO Restaurant VALUES(4,'Spice Route','90 Bloor St Toronto','4165551003','Indian','11:00','23:00',75);
INSERT INTO Restaurant VALUES(5,'Taco Town','15 Front St Toronto','4165551004','Mexican','10:00','21:00',50);
INSERT INTO Restaurant VALUES(6,'Dragon Wok','55 Yonge St
Toronto','4165551005','Chinese','11:00','22:30',65);
INSERT INTO Restaurant VALUES(7,'Burger Barn','300 College St
Toronto','4165551006','American','10:00','22:00',90);
INSERT INTO Restaurant VALUES(8,'Green Garden','88 Bay St
Toronto','4165551007','Vegetarian','09:00','21:00',55);
INSERT INTO Restaurant VALUES(9,'Seaside Grill','12 Harbour St
Toronto','4165551008','Seafood','11:00','23:00',85);
INSERT INTO Restaurant VALUES(10,'Golden Curry','9 Spadina Ave
Toronto','4165551009','Thai','11:00','22:00',60);
CREATE TABLE Customer (
    customer_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT UNIQUE,
    phone_number TEXT,
    loyalty_status TEXT
);
INSERT INTO Customer VALUES(1,'Bob','Smith','bob@email.com','4161110001','Gold');
INSERT INTO Customer VALUES(2,'Alice','Brown','alice@email.com','4161110002','Silver');
INSERT INTO Customer VALUES(3,'John','Lee','john@email.com','4161110003','None');
INSERT INTO Customer VALUES(4,'Maria','Garcia','maria@email.com','4161110004','Gold');
INSERT INTO Customer VALUES(5,'David','Kim','david@email.com','4161110005','Silver');
INSERT INTO Customer VALUES(6,'Sara','Wilson','sara@email.com','4161110006','None');
INSERT INTO Customer VALUES(7,'Omar','Hassan','omar@email.com','4161110007','Gold');
INSERT INTO Customer VALUES(8,'Lina','Patel','lina@email.com','4161110008','Silver');
```

```
INSERT INTO Customer VALUES(9,'Chris','Johnson','chris@email.com','4161110009','None');
INSERT INTO Customer VALUES(10,'Emma','Taylor','emma@email.com','4161110010','Gold');
CREATE TABLE DiningTable (
   restaurant_id INTEGER NOT NULL,
   table_number INTEGER NOT NULL,
   seating_capacity INTEGER NOT NULL,
   availability_status TEXT,
   seating_location_description TEXT,
   PRIMARY KEY (restaurant_id, table_number),
   FOREIGN KEY (restaurant_id)
      REFERENCES Restaurant(restaurant_id)
      ON DELETE CASCADE
);
INSERT INTO DiningTable VALUES(1,1,4,'Available','Window');
INSERT INTO DiningTable VALUES(1,7,6,'Available','Patio');
INSERT INTO DiningTable VALUES(2,3,4,'Available','Center');
INSERT INTO DiningTable VALUES(3,5,4,'Available','Corner');
INSERT INTO DiningTable VALUES(4,2,6,'Available','Center');
INSERT INTO DiningTable VALUES(5,4,4,'Available','Patio');
INSERT INTO DiningTable VALUES(6,6,2,'Available','Window');
INSERT INTO DiningTable VALUES(7,8,6,'Available','Booth');
INSERT INTO DiningTable VALUES(8,1,2,'Available','Window');
INSERT INTO DiningTable VALUES(9,9,4,'Available','Patio');
CREATE TABLE Orders (
   order_id INTEGER PRIMARY KEY AUTOINCREMENT,
   customer_id INTEGER NOT NULL,
   restaurant_id INTEGER NOT NULL,
   reservation_id INTEGER,
   order_date TEXT NOT NULL,
   total_amount REAL,
   order_status TEXT,
   FOREIGN KEY (customer_id)
      REFERENCES Customer(customer_id),
   FOREIGN KEY (restaurant_id)
      REFERENCES Restaurant(restaurant_id),
   FOREIGN KEY (reservation_id)
      REFERENCES Reservation(reservation_id)
);
INSERT INTO Orders VALUES(1,1,1,1,'2026-04-09',100.0,'Completed');
INSERT INTO Orders VALUES(2,2,2,2,'2026-04-10',42.0,'Completed');
INSERT INTO Orders VALUES(3,3,3,3,'2026-04-11',32.0,'Completed');
INSERT INTO Orders VALUES(4,4,4,4,'2026-04-11',50.0,'Completed');
INSERT INTO Orders VALUES(5,5,5,5,'2026-04-12',27.0,'Completed');
INSERT INTO Orders VALUES(6,6,6,6,'2026-04-12',26.0,'Completed');
INSERT INTO Orders VALUES(7,7,7,7,'2026-04-13',48.0,'Completed');
INSERT INTO Orders VALUES(8,8,8,8,'2026-04-13',22.0,'Completed');
```

```
INSERT INTO Orders VALUES(9,9,9,9,'2026-04-14',72.0,'Completed');
INSERT INTO Orders VALUES(10,10,1,10,'2026-04-15',30.0,'Completed');
CREATE TABLE Staff (
    staff_id INTEGER PRIMARY KEY AUTOINCREMENT,
    restaurant_id INTEGER NOT NULL,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    role TEXT,
    hourly_wage REAL,
    FOREIGN KEY (restaurant_id)
        REFERENCES Restaurant(restaurant_id)
);
INSERT INTO Staff VALUES(1,1,'Ross','Miller','Manager',30.0);
INSERT INTO Staff VALUES(2,1,'Anna','Scott','Waiter',18.0);
INSERT INTO Staff VALUES(3,2,'Kenji','Tanaka','Chef',28.0);
INSERT INTO Staff VALUES(4,3,'Marco','Rossi','Chef',26.0);
INSERT INTO Staff VALUES(5,4,'Raj','Patel','Manager',29.0);
INSERT INTO Staff VALUES(6,5,'Carlos','Lopez','Waiter',18.0);
INSERT INTO Staff VALUES(7,6,'Li','Wang','Chef',25.0);
INSERT INTO Staff VALUES(8,7,'Mike','Brown','Supervisor',22.0);
INSERT INTO Staff VALUES(9,8,'Nina','Singh','Host',17.0);
INSERT INTO Staff VALUES(10,9,'Peter','Hall','Waiter',18.0);
CREATE TABLE MenuItem (
    menu_item_id INTEGER PRIMARY KEY AUTOINCREMENT,
    staff_id INTEGER NOT NULL,
    item_name TEXT NOT NULL,
    price REAL NOT NULL,
    availability_status TEXT,
    FOREIGN KEY (staff_id)
        REFERENCES Staff(staff_id)
);
INSERT INTO MenuItem VALUES(1,1,'Mango Juice',5.0,'Available');
INSERT INTO MenuItem VALUES(2,3,'Salmon Sushi Roll',14.0,'Available');
INSERT INTO MenuItem VALUES(3,4,'Spaghetti Carbonara',16.0,'Available');
INSERT INTO MenuItem VALUES(4,5,'Butter Chicken',17.0,'Available');
INSERT INTO MenuItem VALUES(5,6,'Chicken Taco',9.0,'Available');
INSERT INTO MenuItem VALUES(6,7,'Sweet and Sour Chicken',13.0,'Available');
INSERT INTO MenuItem VALUES(7,8,'Double Cheeseburger',12.0,'Available');
INSERT INTO MenuItem VALUES(8,9,'Vegan Salad Bowl',11.0,'Available');
INSERT INTO MenuItem VALUES(9,10,'Grilled Lobster',24.0,'Available');
INSERT INTO MenuItem VALUES(10,2,'Shawarma Platter',15.0,'Available');
CREATE TABLE OrderHandledBy (
    order_id INTEGER NOT NULL,
    staff_id INTEGER NOT NULL,
    PRIMARY KEY (order_id, staff_id),
    FOREIGN KEY (order_id)
```

```
    REFERENCES Orders(order_id),
  FOREIGN KEY (staff_id)
    REFERENCES Staff(staff_id)
);
INSERT INTO OrderHandledBy VALUES(1,2);
INSERT INTO OrderHandledBy VALUES(2,3);
INSERT INTO OrderHandledBy VALUES(3,4);
INSERT INTO OrderHandledBy VALUES(4,5);
INSERT INTO OrderHandledBy VALUES(5,6);
INSERT INTO OrderHandledBy VALUES(6,7);
INSERT INTO OrderHandledBy VALUES(7,8);
INSERT INTO OrderHandledBy VALUES(8,9);
INSERT INTO OrderHandledBy VALUES(9,10);
INSERT INTO OrderHandledBy VALUES(10,2);
CREATE TABLE OrderContainsItem (
  order_id INTEGER NOT NULL,
  menu_item_id INTEGER NOT NULL,
  quantity INTEGER NOT NULL,
  PRIMARY KEY (order_id, menu_item_id),
  FOREIGN KEY (order_id)
    REFERENCES Orders(order_id),
  FOREIGN KEY (menu_item_id)
    REFERENCES MenuItem(menu_item_id)
);
INSERT INTO OrderContainsItem VALUES(1,1,5);
INSERT INTO OrderContainsItem VALUES(2,2,3);
INSERT INTO OrderContainsItem VALUES(3,3,2);
INSERT INTO OrderContainsItem VALUES(4,4,3);
INSERT INTO OrderContainsItem VALUES(5,5,2);
INSERT INTO OrderContainsItem VALUES(6,6,2);
INSERT INTO OrderContainsItem VALUES(7,7,4);
INSERT INTO OrderContainsItem VALUES(8,8,2);
INSERT INTO OrderContainsItem VALUES(9,9,3);
INSERT INTO OrderContainsItem VALUES(10,10,2);
CREATE TABLE Reservation (
  reservation_id INTEGER PRIMARY KEY AUTOINCREMENT,
  customer_id INTEGER NOT NULL,
  restaurant_id INTEGER NOT NULL,
  table_number INTEGER,
  reservation_date TEXT NOT NULL,
  reservation_time TEXT NOT NULL,
  number_of_guests INTEGER,
  reservation_status TEXT,
  seating_start_time TEXT,
  seating_end_time TEXT,
  FOREIGN KEY (customer_id)
```

```
    REFERENCES Customer(customer_id),
  FOREIGN KEY (restaurant_id)
    REFERENCES Restaurant(restaurant_id),
  FOREIGN KEY (restaurant_id, table_number)
    REFERENCES DiningTable(restaurant_id, table_number)
);
INSERT INTO Reservation VALUES(1,1,1,7,'2026-04-09','18:00',5,'Confirmed','18:00','19:30');
INSERT INTO Reservation VALUES(2,2,2,3,'2026-04-10','19:00',3,'Confirmed','19:00','20:30');
INSERT INTO Reservation VALUES(3,3,3,5,'2026-04-11','18:30',2,'Confirmed','18:30','19:30');
INSERT INTO Reservation VALUES(4,4,4,2,'2026-04-11','20:00',4,'Confirmed','20:00','21:30');
INSERT INTO Reservation VALUES(5,5,5,4,'2026-04-12','17:00',3,'Confirmed','17:00','18:30');
INSERT INTO Reservation VALUES(6,6,6,6,'2026-04-12','19:30',2,'Confirmed','19:30','20:30');
INSERT INTO Reservation VALUES(7,7,7,8,'2026-04-13','18:00',5,'Confirmed','18:00','19:30');
INSERT INTO Reservation VALUES(8,8,8,1,'2026-04-13','17:30',2,'Confirmed','17:30','18:30');
INSERT INTO Reservation VALUES(9,9,9,9,'2026-04-14','20:00',3,'Confirmed','20:00','21:30');
INSERT INTO Reservation VALUES(10,10,1,1,'2026-04-15','18:30',2,'Confirmed','18:30','19:30');
PRAGMA writable_schema=ON;
CREATE TABLE IF NOT EXISTS sqlite_sequence(name,seq);
DELETE FROM sqlite_sequence;
INSERT INTO sqlite_sequence VALUES('Restaurant',10);
INSERT INTO sqlite_sequence VALUES('Customer',10);
INSERT INTO sqlite_sequence VALUES('Staff',10);
INSERT INTO sqlite_sequence VALUES('MenuItem',10);
INSERT INTO sqlite_sequence VALUES('Reservation',10);
INSERT INTO sqlite_sequence VALUES('Orders',10);
PRAGMA writable_schema=OFF;
COMMIT;
```

A list of queries that I would like to be answered by the manage system database:

- Which of the Restaurants that have total capacities over 80 people for reservations?
- Which of the following restaurants have “chicken” menu items
- Which of the Restaurants have tables available for a spot for 10 people
- Is the restaurant's highest priced food available for ordering?
- How many restaurants are there in a specific location for Japanese cuisine food?
- What is the highest paid employee in the Mexican restaurants nearby?
- Which hours is this specific restaurant operating within?
- Which branch of the restaurant has the highest rating, and what is its location?
- How many menu/order items does Villa Grill have that is below $10?
- Can I reserve table 1 from “Green Garden” restaurant for 6 to 8 people?

1)

```
SELECT name, total_capacity
FROM Restaurant
WHERE total_capacity > 80;
```

This query searches within the Restaurant table, specifically looking at the restaurants whose total_capacity is voer 80 people; as a result the WHERE clause filters such that only those rows show up and the SELECT function ensures that we retrieve the name and total capacity columns. This helps identify suitable restaurants for larger reservations as they are larger restaurants.

2)

```
SELECT DISTINCT r.name
FROM Restaurant r
JOIN Staff s ON r.restaurant_id = s.restaurant_id
JOIN MenuItem m ON s.staff_id = m.staff_id
WHERE m.item_name LIKE '%Chicken%';
```

This query joins three different tables, in order to locate the specific restaurants that offer chicken-based dishes; it joins Restaurant, Staff, and MenuItem as they have common attributes and can be merged/joined into a single relationship. The LIKE ‘%Chicken%’ is a specific selection condition that searches for the item names containing the word “Chicken”. Also Distinct means that each restaurant can only appear once.

3)

```
SELECT r.name, dt.table_number
FROM Restaurant r
JOIN DiningTable dt ON r.restaurant_id = dt.restaurant_id
WHERE dt.seating_capacity >= 10
        AND dt.availability_status=‘Available’;
```

This query checks which restaurants are capable of seating at least 10 people, by joining the corresponding Restaurants with their DiningTable using the common attribute of restaurant ID used as attribute in both cases. The query as well checks for availability, this indicates potential reservations possible for larger groups.

4)

```
SELECT t.item_name, t.price, t.availability_status
FROM MenuItem t
LEFT JOIN MenuItem b
    ON t.price < b.price
WHERE b.price IS NULL;
```
This query finds the most expensive item available on the menu item list from a restaurant. The query compares each menu item from ‘t’ with every other menu item from ‘b’ in a self join. The condition t.price<b.price checks if there is any other item existing with a higher price. IF there is no such item(its b.price is NULL), then clearly this is the highest priced item.

5)
```
SELECT COUNT(*) AS japanese_restaurants
FROM Restaurant
WHERE cuisine_type = 'Japanese';
```
This query simply counts how many restaurants serve “japanese” cuisine. The COUNT(*) function incrementally totals all the rows matching the clause condition, in this case the Japanese cuisine type.

6)
```
SELECT s1.first_name, s1.last_name, s1.hourly_wage, r.name
FROM Staff s1
JOIN Restaurant r ON s1.restaurant_id = r.restaurant_id
LEFT JOIN Staff s2
    ON s1.restaurant_id = s2.restaurant_id
    AND s1.hourly_wage < s2.hourly_wage
WHERE r.cuisine_type = 'Mexican'
AND s2.staff_id IS NULL;
```
This query joins the Staff s1 and Restaurant table and then does a self join with the s2 employee(other employees) working in the same restaurant(restaurant ID same). The condition s1.hourly_wage<s2.hourly wage checks if another employee exists with a higher wage. IF NOT, the s2.staff_id becomes NULL and we know that the employee on the left is the highest paid employee.

7)
```
SELECT name, opening_hours, Closing_hours
FROM Restaurant
WHERE name = 'Villa Grill';
```
This query retrieves the operating hours for this specific restaurant. The WHERE clause statement defines the Restaurant, and the SELECT statement filters the right columns that we desire, the opening and closing times.

8)
```
SELECT r1.name, r1.address, r1.total_capacity
FROM Restaurant r1
LEFT JOIN Restaurant r2
    ON r1.total_capacity < r2.total_capacity
WHERE r2.restaurant_id IS NULL;
```
This query performs another self-join on the restaurant table with itself. It compares each r1 Restaurant with every other r2 restaurant. The condition r1.total_capacity < r2.total_capacity checks if another restaurant exists with a

larger capacity. IF NOT, then r2.restaurant_id is NULL and we found the highest total capacity.

9)

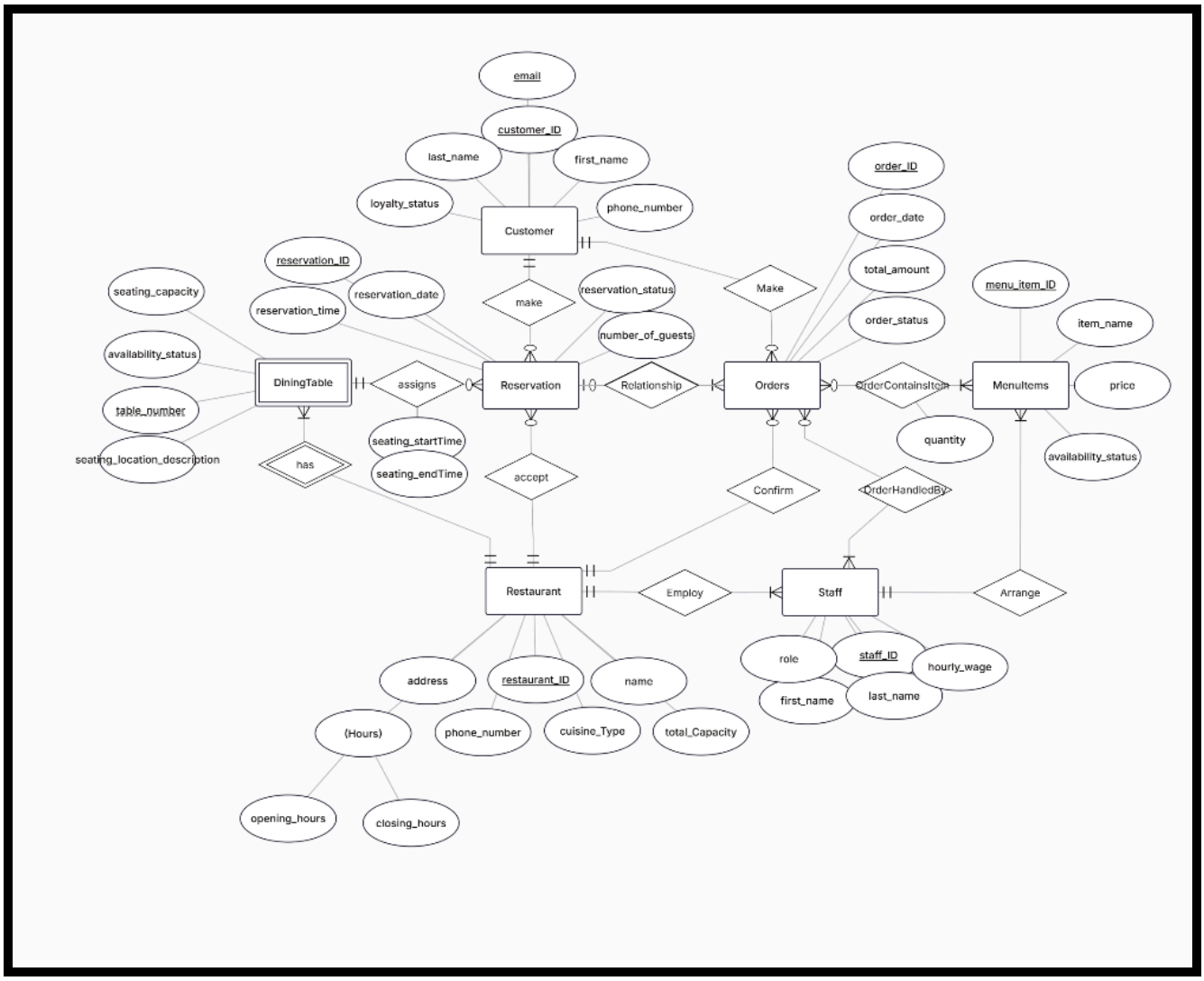
```
SELECT m.item_name, m.price
FROM MenuItem m
JOIN Staff s ON m.staff_id = s.staff_id
JOIN Restaurant r ON s.restaurant_id = r.restaurant_id
WHERE r.name = 'Villa Grill'
AND m.price < 10;
```

This query simply joins MenuItem, Staff and Restaurant tables altogether based on common attributes like staff_id and restaurant_id, such that only the MenuItem and STaff and Restaurant within the same attributes are kept and the rest are dismissed. So we get a table of MenuItems made by specific staff that works with the corresponding restaurant all in the join relationship; it also filters by checking with the name “Villa Grill” and with menu prices under $10.

10)

```
SELECT r.name, dt.table_number, dt.seating_capacity
FROM Restaurant r
JOIN DiningTable dt ON r.restaurant_id = dt.restaurant_id
WHERE r.name = 'Green Garden'
AND dt.table_number = 1
AND dt.seating_capacity BETWEEN 6 AND 8;
```

This query checks if a specific table at the Green Garden restaurant has the required seating capacity or not. The BETWEEN function filters out whether the table can accommodate between 6 to 8 guests. If the query returns the row, that table 1 meets the requirements.


email
customer_ID
last_name
first_name
loyalty_status
phone_number
Customer
reservation_ID
reservation_date
reservation_time
seating_capacity
availability_status
table_number
seating_location_description
DiningTable
has
assigns
seating_startTime
seating_endTime
make
reservation_status
number_of_guests
Reservation
accept
Relationship
Make
order_ID
order_date
total_amount
order_status
Orders
Confirm
OrderContainsItem
quantity
OrderHandledBy
menu_item_ID
item_name
price
availability_status
MenuItems
Restaurant
Employ
Staff
Arrange
address
restaurant_ID
name
(Hours)
phone_number
cuisine_Type
total_Capacity
opening_hours
closing_hours
role
staff_ID
hourly_wage
first_name
last_name